Contents

Adding more features to the REACTJS CRUD APP

Step 1: Backend Authentication Setup

1. Update the MySQL Database

Open http://localhost/phpmyadmin, select sampledb, and run the following SQL command to create a table for your users:

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL
);
```

2. Install Backend Security Packages

In your backend terminal, install the necessary libraries for password hashing and token generation:

```
npm install jsonwebtoken bcryptjs
```

3. Update server.js

We need to add registration, login, and a middleware function to protect your existing product routes. Add the following logic to your existing backend code:

```
const express = require('express');
const mysql = require('mysql2');
const cors = require('cors');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const app = express();
app.use(cors());
app.use(express.json());

const JWT_SECRET = 'super_secret_key_change_in_production';

const db = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'sampledb'
});

// Middleware to verify JWT
const verifyToken = (req, res, next) => {
  const token = req.headers['authorization'];
  if (!token) return res.status(403).json({ message: 'Access Denied: No Token Provided' });

  jwt.verify(token.split(" ")[1], JWT_SECRET, (err, decoded) => {
    if (err) return res.status(401).json({ message: 'Unauthorized: Invalid Token' });
    req.userId = decoded.id;
    next();
```

```
  });
};

// --- AUTH ROUTES ---

app.post('/api/register', async (req, res) => {
  const { email, password } = req.body;
  try {
    const hashedPassword = await bcrypt.hash(password, 10);
    db.query('INSERT INTO users (email, password) VALUES (?, ?)', [email, hashedPassword], (err,
result) => {
      if (err) {
        if (err.code === 'ER_DUP_ENTRY') return res.status(400).json({ message: 'Email already exists'
});
        return res.status(500).json(err);
      }
      res.json({ message: 'User registered successfully' });
    });
  } catch (err) {
    res.status(500).json({ message: 'Server error' });
  }
});

app.post('/api/login', (req, res) => {
  const { email, password } = req.body;
  db.query('SELECT * FROM users WHERE email = ?', [email], async (err, results) => {
    if (err) return res.status(500).json(err);
    if (results.length === 0) return res.status(404).json({ message: 'User not found' });

    const user = results[0];
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });

    const token = jwt.sign({ id: user.id }, JWT_SECRET, { expiresIn: '1h' });
    res.json({ token, email: user.email });
  });
});

// --- PROTECTED CRUD ROUTES ---
// Notice the addition of `verifyToken` in the route definition

app.get('/api/products', verifyToken, (req, res) => {
  db.query('SELECT * FROM products', (err, results) => {
    if (err) return res.status(500).json(err);
    res.json(results);
  });
});
```

```
// Add verifyToken to POST, PUT, and DELETE routes exactly like the GET route above...

app.listen(5000, () => console.log('Server running on port 5000'));
```

Step 2: Frontend Routing & Redux Auth State

1. Install React Router In your React Vite terminal, install the routing package:

```
npm install react-router-dom
```

2. Setup Axios Interceptor (src/api.ts) Instead of manually adding the token to every request, configure Axios to do it automatically. Create this new file:

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:5000/api',
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

Crucial Note(s):

1. ***Go back to src/store/productSlice.ts and replace import axios from 'axios' with import api from '../api'.***
2. ***Change all Axios calls from axios.get(API_URL) to api.get('/products').***

3. Create the Auth Slice (src/store/authSlice.ts)

```
import { createSlice, PayloadAction } from '@reduxjs/toolkit';

interface AuthState {
  token: string | null;
  isAuthenticated: boolean;
}

const initialState: AuthState = {
  token: localStorage.getItem('token'),
  isAuthenticated: !!localStorage.getItem('token'),
};

const authSlice = createSlice({
  name: 'auth',
  initialState,
```

```
  reducers: {
    loginSuccess: (state, action: PayloadAction<string>) => {
      state.token = action.payload;
      state.isAuthenticated = true;
      localStorage.setItem('token', action.payload);
    },
    logout: (state) => {
      state.token = null;
      state.isAuthenticated = false;
      localStorage.removeItem('token');
    },
  },
});

export const { loginSuccess, logout } = authSlice.actions;
export default authSlice.reducer;
```

Update src/store/index.ts to include this new reducer:

```
import { configureStore } from '@reduxjs/toolkit';
import productReducer from './productSlice';
import authReducer from './authSlice';

export const store = configureStore({
  reducer: {
    products: productReducer,
    auth: authReducer,
  },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

Step 3: Auth Components & Routing

1. Create a Protected Route Component (src/components/ProtectedRoute.tsx)

This wrapper will kick unauthorized users to the login screen.

```
import { Navigate, Outlet } from 'react-router-dom';
import { useSelector } from 'react-redux';
import { RootState } from '../store';

export const ProtectedRoute = () => {
  const isAuthenticated = useSelector((state: RootState) => state.auth.isAuthenticated);

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }
  return <Outlet />;
};
```

2. Create the Login Component (src/pages/Login.tsx) We will reuse Zod for form validation here.

```
import React, { useState } from 'react';
import { useDispatch } from 'react-redux';
import { loginSuccess } from '../store/authSlice';
import { useNavigate, Link } from 'react-router-dom';
import api from '../api';
import { z } from 'zod';

const authSchema = z.object({
 email: z.string().email("Invalid email"),
 password: z.string().min(6, "Password must be at least 6 characters"),
});

export const Login = () => {
 const [formData, setFormData] = useState({ email: '', password: '' });
 const [error, setError] = useState('');
 const dispatch = useDispatch();
 const navigate = useNavigate();

 const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  try {
   authSchema.parse(formData);
   const res = await api.post('/login', formData);
   dispatch(loginSuccess(res.data.token));
   navigate('/');
  } catch (err: any) {
   if (err instanceof z.ZodError) {
    setError(err.errors[0].message);
   } else {
    setError(err.response?.data?.message || 'Login failed');
   }
  }
 };

 return (
  <div className="min-h-screen flex items-center justify-center bg-gray-50">
   <div className="bg-white p-8 rounded shadow-md w-96">
    <h2 className="text-2xl font-bold mb-6 text-center">Login</h2>
    {error && <p className="text-red-500 mb-4 text-sm text-center">{error}</p>}
    <form onSubmit={handleSubmit} className="space-y-4">
     <input type="email" placeholder="Email" className="w-full border p-2 rounded"
      onChange={(e) => setFormData({...formData, email: e.target.value})} />
     <input type="password" placeholder="Password" className="w-full border p-2 rounded"
      onChange={(e) => setFormData({...formData, password: e.target.value})} />
     <button type="submit" className="w-full bg-blue-600 text-white py-2 rounded hover:bg-blue-
700">Login</button>
    </form>
```

```
      <p className="mt-4 text-center text-sm">Need an account? <Link to="/register"
className="text-blue-600">Register</Link></p>
    </div>
   </div>
  );
};
```

(You can duplicate this component, change the API endpoint to /register, and redirect to /login to quickly create src/pages/Register.tsx)

3. Wire up src/App.tsx
Rename your original App function to Dashboard (e.g., src/pages/Dashboard.tsx) and add a Logout button to it. Then, update your actual src/App.tsx to handle the routing:

```
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { ProtectedRoute } from './components/ProtectedRoute';
import { Login } from './pages/Login';
import Dashboard from './pages/Dashboard'; // Your previous App component code

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<div>Register Component Here</div>} />
        <Route path="/access-denied" element={<div className="text-center p-10 text-red-600 font-
bold">Access Denied</div>} />

        {/* Protected Routes */}
        <Route element={<ProtectedRoute />}>
          <Route path="/" element={<Dashboard />} />
        </Route>

        <Route path="*" element={<Navigate to="/" />} />
      </Routes>
    </Router>
  );
}

export default App;
```

Step 4: Cypress E2E Testing

1. Initialize Cypress Run the following in your React project terminal:

```
npm install cypress -D
npx cypress open
```

This will launch the Cypress UI. Select **E2E Testing**, let it generate the configuration files, and choose Chrome to start testing.

2. Write Your First Test Create a new file at cypress/e2e/auth_crud.cy.ts. This script will programmatically navigate to your app, log in, and attempt to add a product.

```
describe('Authentication and CRUD Flow', () => {
  const testEmail = 'test@example.com';
  const testPassword = 'password123';

  it('Should log in and add a product', () => {
    // 1. Visit Login Page
    cy.visit('http://localhost:5173/login');

    // 2. Perform Login
    cy.get('input[type="email"]').type(testEmail);
    cy.get('input[type="password"]').type(testPassword);
    cy.get('button[type="submit"]').click();

    // 3. Verify successful redirect to Dashboard
    cy.url().should('eq', 'http://localhost:5173/');
    cy.contains('Product Management').should('be.visible');

    // 4. Test Form Validation (Zod)
    cy.contains('Save Product').click();
    cy.contains('Name must be at least 2 characters').should('be.visible');

    // 5. Add a Product
    cy.get('input[name="name"]').type('Cypress Test Product');
    cy.get('input[name="unitprice"]').type('99.99');
    cy.get('input[name="stocks"]').type('10');
    cy.get('textarea[name="description"]').type('Created via E2E testing.');
    cy.contains('Save Product').click();

    // 6. Verify Product appears in Table
    cy.contains('Product added successfully!').should('be.visible');
    cy.contains('Cypress Test Product').should('be.visible');
  });
});
```